

Datenstrukturen für abstrakte Datentypen (ADTs)

ADTs: Objekte (hier: Schlüssel $\in \mathbb{N}$)
Operationen

Datenstruktur: Implementierung eines ADT

Ziel: Effizienz

1. ADT Stapel (Stack)



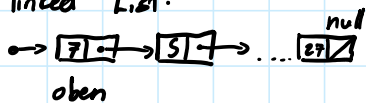
$\text{push}(x, s)$: legt x auf Stapel

$\text{pop}(s)$: entfernt und liefert oberstes Element

$\text{top}(s)$: liefert oberstes Element

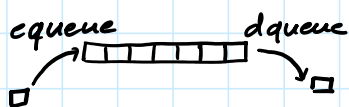
Datenstruktur:

linked List:



alle Ops: $O(1)$

2. ADT Schlange (Queue)



$\text{enqueue}(x, s)$: füge x am ende hinzu

$\text{dequeue}(s)$: entfernt und liefert vorderstes

Datenstruktur:

doubly-linked list

+ Zeigen auf letztes element

alle Ops: $O(1)$

3. ADT Prioritätsschlange (priority queue)

Insert (x, p) : fügt x in P ein

ExtractMax (p) : liefert und löscht Maximum.

Datenstruktur: max heap

Beide Ops: $O(\log n)$: insert $O(\log n)$ versichern

ExtractMax $O(\log n)$: - gebe Max $O(1)$

- restore heap cond $O(\log n)$

4. ADT Wörterbuch (Dictionary)

search (x, W) : ist x in W ?

insert (x, W) : füge x in W ein

remove (x, W) : entferne x aus W

Datenstruktur?

- Sortiertes Array

... 7 | 11 | 28 | 35 ...

suche: $O(\log n)$

insert: suche + verschieben $O(n)$

remove: suche + verschieben $O(n)$

- Linked list (sortiert):

alle Ops $O(n)$

- Max Heap

Suche: $O(n)$

insert: $O(\log n)$

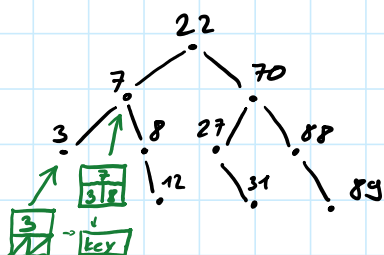
remove: $O(\log n)$

Ziel: alle Ops in $O(\log n)$

Idee: Baum

3 7 8 12 22 27 31 70 88 89

_____ | x | _____



Höhe: $\lceil \log_2(n) \rceil$

im baum kann ich

in $O(\log n)$ suchen



Suchbaum:

für alle x 
in Baum $x' < x < x''$

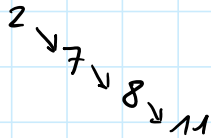
Suche(x, p): if $p = \text{null}$: Misserfolg
else if $p.\text{key} = x$: Erfolg
else if $x < p.\text{key}$: suche($x, p.\text{left}$)
else: Suche($x, p.\text{right}$)

Einfügen(x, p)

zuerst Suche(x, p)

ersetze Blatt durch 

2, 7, 8, 11, 12



Problem: wie halten wir höhe klein?

Rep Suchbaum: n Knoten

Suche: $O(h)$ Höhe h

Einfügen: $O(h)$

Entfernen:

Entferne(x, p):

zuerst Suche(x, p)



Fall 1:  : einfach löschen

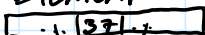
Fall 2:  Teilbaum umhängen

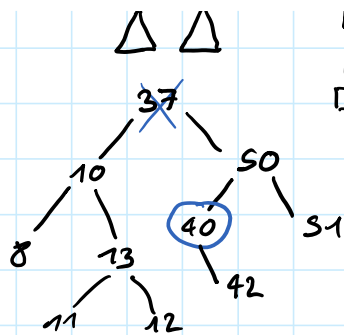
Fall 3:



27

Ersetze x durch anderes Element





Ersetze x durch anderes

Element



Ersetze mit nächst grösserem: symmetrischer Nachfolger
gehe in rechtem Teilbaum immer links

tausche x mit Sym NF

↳ löschen dann wie Fall 1 oder 2

Entfernen $O(h)$

bleibt: erhalte h dicht an $\log n$

↳ $h = \Theta(\log n)$

Idee 1: Erhalte



perfekt
balanciert

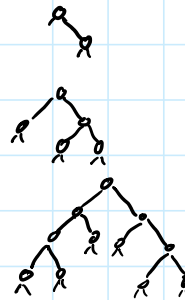
→ schwierig

Idee 2: erhalte folgende
Struktur bedingung:



AVL bedingung
für alle Knoten:
 $|h_l - h_r| \leq 1$

Beispiele



Höhe eines AVL Baums?

Satz: Binärbaum

n Knoten $\Leftrightarrow n+1$ Blätter.

Idee: untere Schranke für die
Anzahl Blätter.

$MB(h) =$ Mindest Blatzzahl
für Höhe h .

Anzahl Blätter $\geq MB(h)$

$n \geq MB(h) - 1$

$MB(1) = 2$

$MB(2) = 3$

$MB(h) =$

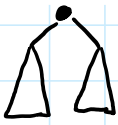


ein TB: $h-1$

anderer: $h-1 \vee h-2$

(Anzahl blätter bei algo $n+1$)

(daher $MB \leq n+1$)

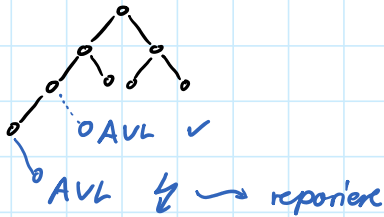
$$MB(h) = \begin{cases} \text{ein LB: } h-1 \\ \text{anderer: } h-1 \vee h-2 \end{cases}$$


$$\begin{aligned} MB(h) &= MB(h-1) + MB(h-2) \\ &= \text{Fib}(h+2) \\ &= \Theta\left(\underbrace{\left(\frac{1+\sqrt{5}}{2}\right)^h}_{1.6 \dots h}\right) \end{aligned}$$

$$n \geq 1.6^h$$

$$h \leq 1.44 \cdot \log(n) \leq O(\log n)$$

Einfügen $\Rightarrow$ einfügen
 + evtl reparieren
 (AVL bedingung erhalten)



Wörterbuch

Baum

VI

Suchbaum alle OPs in $O(h)$

VI

AVL Baum $h = 1.44 \cdot \log_2(n)$

$\Rightarrow$ alle OPs in $O(\log n)$

Einfügen:



wenn TBs wachsen

Wir merken uns in jedem Knoten p :

$$\text{bal}(p) = h_r - h_l$$

$$\text{bal}(p) =$$

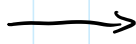
-1: linker TB höher

0: gleich hoch

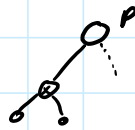
1: rechter TB höher

Einfügen von x (Annahme Links) (wird links eingefügt)

vorher



nachher



Fälle:

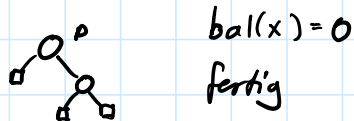
(vorher) 1. $bal(p) = -1$: geht nicht

2. $bal(p) = 0$: $bal(p) = -1$

↳ TB gewachsen $upin(p)$ $bal(x) = 0$

(Repariere AVL condition in Knoten dauer)

3. $bal(p) = +1$: $bal(p) = 0$



$bal(x) = 0$
fertig

keine reparatur nötig.

(1-3) p, x sind fertige AVL bäume

Bei Aufruf von $upin(p)$

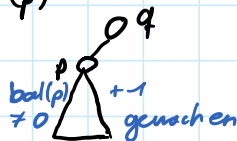
gilt (invariante):

- TB p ist gewachsen

- p ist vorgänger

- $bal(p) \neq 0$

$upin(p)$



Fälle: (wie vorher)

1. $bal(q) = +1$: $bal(q) = 0$

fertig

2. $bal(q) = 0$: $bal(q) = -1$

↳ TB gewachsen

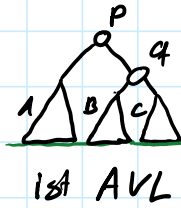
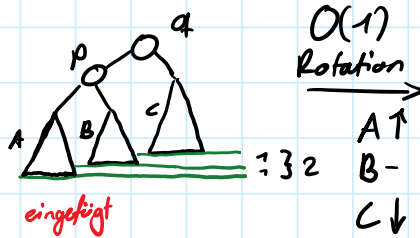
$upin(q)$

3. $bal(q) = -1$: q kein AVL mehr
($bal = -2$)

2 Fälle: $bal(p) = +/- 1$ (nach initialem Einfügen)

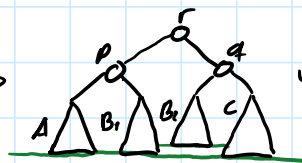
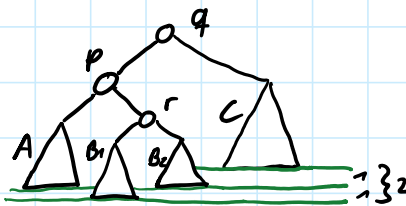
3a) $bal(p) = -1$

Suchbaum ✓



$bal(p) = 0$
 $bal(q) = 0$
TB(p) ist gleich hoch, wie TB(q) vor einfügen

3b) $bal(p) = +1$



$bal(r) = 0$
Suchbaum ✓ $bal(p) = 0$
 $bal(q) = +1$

Einfügen $O(h)^{AVL} = O(\log n)$